What survives on a small team's rig: re-testing 21 robot-learning papers with cheap hardware and little data

Ravindu Somawansa
Shisso Robotics
ravindu.somawansa@shissorobotics.com

Abstract. Most of the robotic papers are either tested in sim or on expensive rigs. From our tested papers, 16 out of 21 were done in simulation or on rigs above $20k, and only 1 on a rig under $5k. But the question is, does a claim still validate for smaller teams with cheaper hardware, less data and shorter tasks? This is what we want to answer in this paper. For that we re-tested 36 claims out of 21 papers on our cheap hardware, an Nvidia Jetson Orin NX 16GB and a robot arm SO-101 (about €1,500). We used the same protocol for each claim: a bar fixed before any testing, and the cheapest baseline tested first. Out of the 36 claims, only 11 had a cheap baseline to test against, and among them only 4 are worse than this baseline. Among those 4, LeWorldModel, DINO-WM and ContactGuard are clean fails, and Robometer's loss comes from how its dataset ends the episodes. LeWorldModel and DINO-WM picked the chunk the arm really executed less often than a baseline using the actions alone, at every distance we tested, and ContactGuard predicted failures worse from its imagined latent than from the current one. Our results: 12 claims hold, 11 fail using the paper's method, 4 fail on our variation, 2 on our inference engine and 1 on a task outside the paper's regime. Most of the failures come as much from the papers' methods as from our short tasks, small data and our own variants, and for each we explain why. The code, data and all numbers are public and can be replicated.

Code and rows: `github.com/ravediamond/paper-register-reproducibility` **Reproduce:** `make check`. Every number carries its source file and protocol commit.

1 Introduction

Out of the 21 papers we tested, 16 of them are done in simulation or on expensive rigs above $20k, and only 1 on a rig under $5k. This means small teams can either trust the paper or re-test the claim on their own hardware. Both of them are costly, either when validating or debugging if it does not work. The question really is, do the papers' claims still hold on cheap hardware, with little data and short tasks? And this is the environment small teams or solo founders have.

To answer this question, we re-tested each claim on our own cheap rig. We used the same testing protocol each time: a bar fixed before any test, and the cheapest baseline (low cost or free) run first on the same data, to filter out claims early before investing in them.

We found out that this cheap baseline was really useful to validate claims. For example with LeWorldModel, the goal was to predict the next state of the arm and plan with it. For that we had a first baseline where it needed to beat a copy-last method, and it passed (error 0.31 vs 0.89). But for ranking the chunk the arm really executed, the model scored worse than using the actions alone (0.60 vs 0.68–0.90). Without that second baseline, we would have called the paper a success.

Out of the 36 claims, only 12 hold on our rig. The failures are split between the paper's method failing (11), our variant of the method failing (4), our inference engine (2) and one outside the paper's regime. But, and it is important to say it again, a fail on our rig does not mean the paper is wrong: it just does not work on our specific setup and data.

2 Rig and data

Here's the rig we used for re-tests:

- A robot arm SO-101: 5 joints + gripper, 1 wrist camera and 1 front-top camera
- Leader + follower arm: all teleoperation done using the leader arm
- A Nvidia Jetson Orin NX 16GB: runs the inference
- Cloud GPU instances for training only

The estimated cost of the whole rig is about €1,500, plus under $50 of cloud GPU (for all re-tests).

Here's the data we used:

The teleoperated data (HF dataset: so101_bowls_all_trimmed):

- 2 short tasks: put one bowl into another
- 182 episodes, 61,462 frames at 30 fps, about 34 min
- split: 157 episodes in the train set, 25 in the validation set
- mostly used for the world models and the VLA policies

Some policy rollouts:

- 69 episodes: 44 successes, 25 failures, labelled by hand by the operator
- main set for most monitor tests: 50 episodes (42 successes / 8 failures), leaving out the episodes of one policy family (TurboVLA's)
- FAIL-Detect / SAFE: the 44 successes split in half, 22 to calibrate and 22 to test, over 20 random splits
- a separate set of 88 of our own episodes, top camera only, to check the reward model directly on our rig

The ArmnetBench dataset (public, HF dataset: armnet/armnetbench_v01_lerobot_so101) [14]: 2,099 labelled SO-101 rollouts from 7 policies, used for the reward models and judges.

The timing set:

- 20 recorded episodes
- 254 chunk boundaries

This is used for analysing and measuring chunking and latency.

But this data has limits that explain some of the failures:

- Short tasks only: each episode is about 11 s
- No multi-step, memory-based or long-horizon tasks
- Everything has been done in one place, with one robot and one task family, which means this may not translate to real-world usage of such tasks.
- All the rollout outcomes have been judged by one human, so they are subjective (not blind).

3 What we tested and how

There are 4 kinds of papers we picked for re-tests (see Table 1 for details):

- World models (6 papers): the goal is to learn a latent from the cameras to predict or plan. In this case, the question is "does the latent beat a free signal?"
- Failure monitors and judges (8 papers): the goal is to detect if an episode fails, or how far it has progressed. In this case, the question is "does the monitor beat a trivial signal?"
- VLA policies (3 papers): the goal is to follow and execute an instruction on the arm.
- Chunking and edge inference (4 papers): the goal is to improve how action chunks are made and joined, by making it smoother or faster.

All the papers were chosen because we judged that they could reasonably run on our rig, which filters out many papers from the get-go.

We used 2 rules to re-test the claims.

Rule 1: the bar to achieve to validate the claim needs to be written from the paper and its code, not after running the claim once. This avoids contaminating the bar with our own results. Where the paper gives a number, the bar comes from it; otherwise the bar is ours.

Rule 2: we create and launch the cheapest baseline first, on the same data. It is built from cheap things that can be used on our tasks, like the last frame, the joint values or the episode length. If the method loses to it, the bar is not useful. For example, does a failure monitor catch failures better than directly checking the episode length?

There are only 4 verdicts possible for each re-test: holds, meaning the bar is met; fails, so the bar is missed; not proven, because we cannot decide considering our rig and context; and not tested, not run yet (because of time to test, cost, ...). A paper is considered failed if any of its claims fails. We decided that because, for us, a paper that is only half successful is considered failed. All the failures can be categorized in 4 categories:

- the paper's method failed
- our variant of the method failed (because we could not use the paper's method directly)
- the limitation of our inference engine
- or a task that goes outside the paper's scope

We had 2 cases of weaker bar: for SmolVLA and VLA-JEPA, we had no numeric bar, so instead we launched 10 trials judged by a human. 9 out of 10 were a success, which means a true rate anywhere from 60% to 98% (Wilson interval, 95%).

4 Results

Table 2 and Figure 1 show the verdicts, and Table 1 breaks them down by kind of paper. Out of the 36 tested claims, 12 hold, 18 fail (11 using the papers' methods, 4 using our variant, 2 on our inference engine and 1 outside the paper's scope), 3 are not proven and 3 not tested. At the paper level, 4 are validated, 14 fail and 3 are not proven.

The pattern by kind looks revealing (all 8 failure monitors and judges have failed, all 3 VLA policies reproduce) but is actually limited. 3 out of 8 monitors used our own variant, so they cannot directly serve as evidence. As for the VLAs, for 2 of the 3 there were no numbers to use as bars, so we just ran trials and recorded the number of fails.

5 The cheapest baseline

The cheapest baseline is a test we get for cheap or near nothing on the same data. No training is needed and it should run in a few minutes (see Table 1). For example copy-last (predict the next frame as the current one), episode length (use the episode length to determine the outcome of an episode) or the commanded joints (select the action chunk whose commanded joints are closest to the joints measured at the goal). For these tests, we do a paired bootstrap: both the paper's method and the baseline are scored on the same data, we resample the episodes 2,000 times and compare the scores with a 95 % interval. Out of all claims, 11 had a baseline test and 5 lost to it, 4 of them with an interval below zero:

- LeWorldModel and DINO-WM vs actions alone. The goal is for the method to pick the chunk the robot really executed. The baseline just picks the chunk whose commanded joints are closest to the goal. LeWorldModel picks it at most 60 % of the time and DINO-WM slightly more, but the baseline picks it 0.68–0.90 of the time (depending on how far ahead it is looking), a gap of 23 to 37 points for LeWorldModel and 10 to 28 for DINO-WM, at every distance. The baseline is strong because our arm goes almost exactly where it is commanded, so the commanded joints already say where it will end up.
- ContactGuard vs the current latent. The goal of the model is to predict failure from the imagined future latent. The baseline just uses the current latent to predict failure: 0.82 vs 0.70 AUROC. The gap is 0.12, and the interval $[-0.24, -0.01]$ is below zero.
- Robometer vs episode length: the goal of the method is to judge the success or failure of the task. But it loses to the baseline, which only checks the duration of the episode (0.938 vs 0.818 on ArmnetBench). This is a problem we discovered on ArmnetBench, where the operators stop a success when it is done but let a failure run on.

The last one, adaptive chunking, was actually considered uncertain because the confidence interval $[-0.17,$

Table 1: The four kinds of papers, what we test in each, the cheapest baseline we build for it, and the verdicts.

kind of paper	papers	what we test	cheapest baseline	verdicts
world models	6	predict the next state; rank or plan actions	copy-last (next state = current state); the commanded joints or the actions alone	1 reproduce, 4 fail, 1 not proven
failure monitors and judges	8	catch failures; track progress	episode length; the same model with shuffled actions	8 fail
VLA policies	3	succeed on our arm	none: success over 10 trials judged by the operator, no numeric bar fixed before	3 reproduce
chunking and edge inference	4	smoother joins between chunks; time per chunk	the same policy without the trick; the real-time budget on the Orin	2 fail, 2 not proven



Figure 1: the 36 paper claims by verdict: 12 reproduce, 11 fail with the authors' method on our data, 7 fail only in our variant of the method, because of our inference engine or outside the paper's regime, and the rest are open.

Table 2: Verdicts by paper and by claim. A paper fails if any of its claims fails; failures are split by whose method failed.

verdict	papers	claims
reproduces	4	12
fails, authors' method	9	11
fails, our variant	3	4
fails, our engine	1	2
fails, outside the paper's regime	1	1
not proven	3	3
not tested	0	3

+0.06] contains zero, meaning in some resamples it is better than the fixed chunk length and in others it is worse. Also, with only 20 episodes, there is too little data to decide, so we do not count it as a real loss.

As for ContactGuard's paper results, we found the opposite. The goal of ContactGuard is to imagine the future latent with the world model and predict failure or success from this imagined latent. The compared baseline just encodes the current frames as a latent, skips the imagination and uses the same probe as with imagination. So the question becomes: is it really useful to imagine the future to determine failure or success? From the paper, it is a yes (AUROC 0.98 vs 0.84), but a no from our tests (0.70 vs 0.82).

One thing we discovered is that the choice of the baseline decides the verdict of the claims. For example, LeWorldModel predicts the next latent much better than the copy-last baseline (0.31 vs 0.89), so this claim holds. But on the test of picking the executed chunk, the model performs worse than the commanded-joints baseline. Basically, predicting well is not equal to planning well, and with just the first baseline we would have concluded that the claim was a pass.

But baselines are not equal; some are extremely strong because of a quirk. For example, on ArmnetBench, successes are stopped when they are done and failures run on, so the episode length baseline is very strong. But on our own episodes, which do not have this quirk, it is near chance (0.521). This also means that analysing the baseline results rigorously is very important, as a baseline could be far too strong.

In our case, baselines were particularly strong because our tasks are short and simple (for any task only 1 or 2 joints usually move). On long-horizon tasks, a learned latent model has more to give. Where the papers report their own cheap baseline, the methods still beat it on our data: PACT-VLA (against a random pick) and HWM (against net joint displacement).

In practice, a baseline needs to be crafted with care, but after that, if the model cannot beat it, then we can stop. If it beats it, then it also needs to beat the bar to be recognized as a validated claim.

6 Can the model run on the hardware fast enough?

Time to generate a chunk is critical for real-world robotics. The generation of a chunk should at least be done before the previous chunk is executed, and that is the bare minimum. The problem is that when the paper is tested in sim or on a big GPU (cloud or desktop GPU), the latency numbers do not hold on small hardware. When testing papers we use our edge GPU (the Orin), which is also the main inference system, so a latency claim fails if the method cannot run on it fast enough.

So what fits on our Orin, and what made it better?

- SmolVLA: 974 ms in PyTorch (too slow), but 238 ms with vla.cpp,

Table 3: The paper claims we compared with a cheap baseline, on the same data.

method	baseline (what is measured)	method	baseline	outcome	paired difference, positive = method better [95 % interval]
LeWorldModel	copy-last (latent error at 30 steps, lower is better)	0.31	0.89	beat	+0.58 [+0.51, +0.66], beats, on the re-run (unit = window)
LeWorldModel	actions alone (executed chunk ranked first)	0.60	0.68–0.90	lost	worst case d = 10: −0.37 [−0.42, −0.32]; best case d = 5: −0.23 [−0.29, −0.18], loses
DINO-WM	actions alone (executed chunk ranked first)	0.61	0.68–0.90	lost	worst case d = 15: −0.28 [−0.31, −0.25]; best case d = 5: −0.10 [−0.15, −0.04], loses, on the re-run
RoboJEPA	copy-last (9-step latent error)	11.8 % below copy-last	–	beat, under its bar of 15 %	+0.12 [+0.11, +0.13], beats
ContactGuard	the current latent (failure AUROC)	0.70	0.82	lost	−0.12 [−0.24, −0.01], loses
ReAP	shuffled actions (failure AUROC)	0.789	0.708	beat	+0.08 [−0.05, +0.20], unclear, on the re-run
Robometer	episode length (failure AUROC, ArmnetBench)	0.818	0.938	lost	−0.12 [−0.15, −0.09], loses
HWM	net joint displacement (cosine to the macro-action)	0.608	0.551	beat	+0.06 [+0.03, +0.08], beats
Adaptive chunking	fixed chunk length (boundary jump, lower is better)	0.973	0.940	lost	−0.03 [−0.17, +0.06], unclear
PACT-VLA	a random pick (error of the executed window)	−0.27° against random	–	beat	+0.27 [+0.09, +0.44], beats
REMORA	per-frame readout (readout flips)	0.70 × per-frame	–	beat, under its bar of 0.5 ×	ratio 0.70 [0.48, 0.94], beats

the inference engine we used in our tests, which was 4× faster than PyTorch. So the choice of the inference engine was really useful.

- VLA-JEPA: 355 ms per chunk.
- Fewer denoising steps (horizon × depth): 2 steps instead of 10 saves only 17 % (196 vs 238 ms), because the vision part alone takes 151 ms.
- Real-time chunking: the smoothing trick costs only 0.4 ms.

What did not fit?

- RoboJEPA's encoder: 526 ms vs 200 ms required.
- Methods that sample many chunks and pick one get expensive without batching, because our engine runs the samples one after another:
  - Adaptive chunking: 4,751 ms per decision vs a 298 ms budget, since it pays for 20 full passes.
  - PACT-VLA: about 1,200 ms per cycle (estimated) (37 control ticks vs 8).

These two fail on our engine, not necessarily on the method: with batching, they might fit.

So the lesson is that latency is one of the simplest checks, and it tells you cheaply whether there is a need to invest in a claim. And of course it is possible to improve it, for example by choosing the right inference engine, using batching or lowering the denoising steps, but the hard limit that needs to be met is the time to execute one chunk. And it is testable without any training.

7 Why they failed here: the method, our data, or our setup

In Table 4, you can see which method failed, and what could have provoked it on our side. In most of the failures, we have at least one cause on our side. So a fail just means a fail on our rig and on our data, which means that if a team has more data or a better rig, the paper may still work.

One of the limits of our data is its size and simplicity. It is 157 training episodes, with basically one task of about 11 s. Most of the papers use hours of varying tasks. For example, the Hierarchical World Model paper uses 130 h of data against our 0.6 h. Because the tasks are so simple, the learned latents cannot give much more than using the joint commands directly. For VLA-based papers, because our task was so simple, there was little uncertainty, the policy succeeded most of the time, and the optimization papers had little to work with (86 % of chunk lengths at the minimal value for the adaptive chunking paper).

Another problem is that we had too few failures, only 8 in 50 episodes, which makes verdicts on bars about catching failures (like 5 out of 8) very coarse.

One thing we needed for some papers was to adapt the paper's method to fit our rig or data. Of course we used the original one if possible. 4 methods were only fitted to our rig: ContactGuard, VLA-Corrector, ReAP and SURVEYOR's threshold. They count as the authors' method. The differences are in Table 4. 3 retests had real variants:

- FAIL-Detect and SAFE: their code needs the policy's internal features, and only 24 successes and 6 failures of ours could be re-run through it. So we used our world model's surprise as the score.
- SURVEYOR's monitor: the drafter isn't released, so we used our own model's imagination.
- REMORA: its progress model was trained on 8 H200 GPUs, so we tested only its readout.

In these cases, the failures are failures of our variant,

on our rig and data, not in the context of the papers.

Some retests we did only offline, not on the arm, as it was simpler and faster and we could still do a good retest. For example:

- For planning: we ranked recorded chunks; the robot didn't execute the plan.
- For chunking: we measured smoothness on recorded episodes.

So for these, we can't say with full proof whether the robot would do the task better or worse.

One more point to add is that if a paper failed because of our rig or our data or both, this also means that it can fail on other similar low-cost setups. This suggests that some of the claims need either large compute or large data to work, and so cannot be used immediately by a small team without sufficient resources.

8 Related work

Some works already evaluate policies on the same arm as ours. One is a real-world VLA benchmark that also lists the kinds of failures and recoveries [13], and another is ArmnetBench, a fleet of SO-101 cells that evaluates policies at scale [14]. In simulation, vla-eval runs VLAs across 14 benchmarks [8], and another benchmark looks at the quality of the demonstrations [20]. All of these rank policies against each other. What we do is different: we take the claim a paper makes about its method and test it against a bar fixed before the test, on one cheap rig. The closest work to ours is RoboSeed [26], which scores VLA and world-action models on five dimensions, including reproduction evidence and how affordable the deployment is. But its authors say that the score is not a probability that a result is correct. On the reproduction side, ReproBridge [27] has coding agents re-run 1,029 tasks from 36 papers with the papers' own code, and they succeed on about 76 % of them with the raw code and 93 % with a cleaned-up version of it. Their question is whether you get the same number with the same setup. Ours is whether the claim still holds on a different and cheap rig, and for each claim we give one verdict, the data behind it and why we think it failed.

9 Conclusion

In conclusion, we re-tested 36 claims from 21 papers, with our €1,500 rig and under $50 of cloud GPU. The results: 12 claims held, 11 failed with the paper's method, 7 more failed on our own variant or setup, and the rest are uncertain. But the failures are failures on our specific rig and data, or even on our method variation, not the paper's failure. Some are because our data is too simple or short, others because our edge GPU is not powerful enough, but these are the limitations any small team with little resources will have. The advice that we can give then is to follow the experiment method we used in all our re-tests: create a bar from only the paper and its code, and find the cheapest baseline that can be tested on your data without any training and really fits what the paper shows. Then first test if the method passes the baseline (a fast and cheap stop signal), and then see if it passes the bar. All our re-tests are public, and every number can be rebuilt and tested from our repository. Authors who think the failures are ours are welcome to tell us, and we will gladly correct them.

References

[1] DINO-WM: World Models on Pre-trained Visual Features enable Zero-shot Planning, 2024. arXiv:2411.04983.

[2] Can We Detect Failures Without Failure Data? Uncertainty-Aware Runtime Failure Detection for Imitation Learning Policies, 2025. arXiv:2503.08558.

[3] SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics, 2025. arXiv:2506.01844.

[4] Real-Time Execution of Action Chunking Flow Policies, 2025. arXiv:2506.07339.

[5] SAFE: Multitask Failure Detection for Vision-Language-Action Models, 2025. arXiv:2506.09937.

[6] VLA-JEPA: Enhancing Vision-Language-Action Model with Latent World Model, 2026. arXiv:2602.10098.

[7] Robometer: Scaling General-Purpose Robotic Reward Models via Trajectory Comparisons, 2026. arXiv:2603.02115.

[8] vla-eval: A Unified Evaluation Harness for Vision-Language-Action Models, 2026. arXiv:2603.13966.

[9] LeWorldModel: Stable End-to-End Joint-Embedding Predictive Architecture from Pixels, 2026. arXiv:2603.19312.

[10] Hierarchical Planning with Latent World Models, 2026. arXiv:2604.03208.

[11] Adaptive Action Chunking at Inference-time for Vision-Language-Action Models, 2026. arXiv:2604.04161.

[12] When Does LeJEPA Learn a World Model?, 2026. arXiv:2605.26379.

[13] Benchmarking Vision-Language-Action Models on SO-101: Failure and Recovery Analysis, 2026. arXiv:2606.08881.

[14] ArmnetBench v0.1: Parallel Real-World Evaluation of Manipulation Policies on a Low-Cost Arm Farm, 2026. arXiv:2607.24481.

Table 4: Each failed or not-proven paper, whose method failed, what we measured, and what on our side may explain it.

failed paper	whose method	our result	what may be on our side
LeWorldModel	authors'	planning rank 0.60, action-only 0.68–0.90	one short task, 157 episodes, one seed; offline ranking only
DINO-WM	authors'	planning rank 0.61, action-only 0.68–0.90	same data as LeWorldModel; one training
RoboJEPA	authors'	11.8 % below copy-last, bar 15 %	near miss; the 1B model reached 15.8 % on 30 clips only
H-JEPA	outside its regime	progress ρ −0.73, bar −0.80; false stalls 52 %	our task is outside the paper's regime
HWM	not proven	not proven: high level worse at 2 s	0.6 h of one task against their 130 h
ContactGuard	authors'	AUROC 0.70, current latent 0.82	mixed policies and 2 cameras against one policy and 3 cameras
FAIL-Detect and SAFE	our variant	9.1 % of successes flagged at α 5 % (papers' band)	our score, not theirs; 11 + 11 successes to build the band
ReAP	authors' method, reduced	2 of 8 failures caught	Stage 1 and 2 rebuilt from the paper, smaller DINOv2, one camera, mostly other tasks; one seed of five; 8 failures
VLA-Corrector	authors'	1 of 8 caught at 3 of 42, AUROC 0.53	30 training rollouts per fold against their 120 episodes; our onset labels, no stage head; 8 failures
Robometer	authors'	AUROC 0.818, length 0.938	zero-shot only; on ArmnetBench failures run longer than successes (on our rollouts length is near chance)
SURVEYOR	authors' (threshold); our variant (monitor)	threshold 1.249 against 0.20; 3 of 8	their drafter not released; 8 failures
REMORA	our variant	readout flips 0.70 of the per-frame readout	hand-written progress atoms
Adaptive chunking	authors' (latency: our engine)	4,751 ms per decision; length 2 in transport and at the grasp	sequential engine, no batching; task at ceiling for our policy
PACT-VLA	our engine	37 ticks of delay against 8	check time projected; sequential engine
Real-Time Chunking	not proven	not proven: 71–73 % smoother, no arm run	offline smoothness only
Horizon × depth	not proven	not proven: 0.78, bar 0.70	one policy, one task; arm test not run

[15] ContactGuard: Pre-Contact Execution Monitoring with Action-Conditioned Latent World Models, 2026. arXiv:2608.13438.

[16] VLA-Corrector: Stage-Aware Observable State Understanding for Prompt-Based Closed-Loop Recovery of Vision-Language-Action Policies, 2026. arXiv:2609.06508.

[17] H-JEPA: End-to-End Learning of Hierarchical World Models for Visual Planning, 2026. arXiv:2610.06805.

[18] RoboJEPA: Scaling Robotic Latent World Models, 2026. arXiv:2610.10515.

[19] ReAP: From Visual Failure Recognition to Action-Conditioned Risk Prediction for Runtime Failure Monitoring, 2026. arXiv:2610.16309.

[20] What Matters in Quality-Aware VLA Pretraining: A Benchmark and Empirical Study, 2026. arXiv:2610.18869.

[21] REKI: Registering Knowledge in VLAs, 2026. arXiv:2610.22178.

[22] PACT-VLA: Speculative Action-Chunk Verification with Pre-Commit Observations, 2026. arXiv:2610.28019.

[23] Coupling Execution Horizon and Sampling Depth in Flow-Matching VLA Policies, 2026. arXiv:2610.32102.

[24] Tracking Task Progress Through Errors and Recovery:A Benchmark and Recurrent Distributional Model, 2026. arXiv:2610.39125.

[25] SURVEYOR: Certified Speculative Plan Consumption for Latent World-Model Planning. NeurIPS 2026 workshop, 2026. neurips.cc/virtual/2026/175857.

[26] Xin Ma et al. RoboSeed: Multidimensional Evaluation of Vision-Language-Action and World-Action Models: Capability, Resource Affordability, and Research Evidence, 2026. arXiv:2610.40026.

[27] Yubo Wang, Rui Meng, Jaehyun Nam, Tomas Pfister, and Jinsung Yoon. ReproBridge: Building Reproducibility into ML Research Artifacts, 2026.

Appendix: every paper

paper	their rig	claim tested	bar	our result	verdict
LeWorldModel [9]	sim	A small JEPA world model learns the arm's dynamics well enough to plan with it.	Dynamics: beat copy-last, every corruption ≥ 1.3 at $K = 30$. Planning: rank the executed chunk first among 8 in ≥ 0.8 of anchors.	dynamics 0.31 vs 0.89 copy-last (reproduces); planning top-1 0.60 vs 0.68–0.90 for an action-only baseline (fail)	fail (authors' method)
DINO-WM [1]	sim	A predictor on frozen DINOv2 patches plans better than one on a learned latent.	G1b top-1 ≥ 0.8 at some target distance.	arm-pose R^2 0.92 (vs 0.57 for W1); G1b top-1 0.55–0.61; no-model baseline 0.68–0.90	fail (authors' method)
RoboJEPA [18]	>$20k	Pretrained RoboJEPA predicts my latents, uses the actions, and runs on the robot's computer.	9-step L1 ≥ 15 % below copy-last (300M) and ≥ 5 % below shuffled actions; 2-view encoder ≤ 200 ms on the Orin.	11.8 % below copy-last (300M; 1B 15.8 % on 30 clips); 9–10 % below shuffled; encoder 526 ms on the Orin NX	fail (authors' method)
H-JEPA [17]	sim	A slow second level makes latent distance track task progress.	Progress monotonicity Spearman $\rho \leq -0.80$; false stalls ≤ 10 %.	level-2 ρ -0.73; false stalls 52 %	fail (outside the paper's regime)
ContactGuard [15]	$5–20k	An event-anchored probe on imagined latents predicts grasp failure better than the current latent.	AUROC ≥ 0.85 and ≥ 0.05 above the current latent.	0.70 imagined vs 0.82 current	fail (authors' method)
FAIL-Detect and SAFE [2] [5]	>$20k	A conformal band on a learned failure score keeps false alarms at α while catching failures.	Successes flagged $\leq 1/20$; failures flagged ≥ 50 %.	9.1 % of successes flagged at $\alpha = 5$ %; 24 % of failures caught (first band built wrong: 22.7 % / 46 %)	fail (our variant)
ReAP [19]	$5–20k	A risk head on an action-conditioned latent warns before failures.	$\geq 5/8$ failures caught at $\leq 1/20$ false interventions; action-conditioned iff true $\geq$ shuffled + 0.05.	v1 AUROC 0.93 came from time and length; v2 0.789 vs 0.708 shuffled, 3/8 at 5/42; with its Stage 1 (one seed): 2/8 at 1/42	fail (authors' method)
VLA-Corrector [16]	sim	A verifier on observable history flags a failing rollout early enough to correct it.	$\geq 5/8$ failures at $\leq 1/20$ false interventions.	the paper's Learned Verifier on our rollouts, built and trained as the paper describes (54 RGB features per view, joint positions, executed actions, its labels, optimiser and gate): 1/8 caught at 3/42 false interventions, AUROC 0.53; the earlier run with the same input and our own training: 2/8 at 3/42, AUROC 0.59; cosine drift of the W1 latent 0/8, AUROC 0.30	fail (authors' method)
Robometer [7]	>$20k	A 4B reward model judges SO-101 rollouts zero-shot.	Pooled AUROC ≥ 0.85 and within-policy ≥ 0.8 on ≥ 5 of 7 policies.	pooled AUROC 0.818 on 2099 ArmnetBench rollouts; episode length alone 0.938	fail (authors' method)
SmolVLA [3]	<$5k	A 450M VLA succeeds on an SO-101 task and can run on low-cost hardware.	10-trial operator-judged gate (no numeric bar); chunk inference within the async budget (667 ms).	9/10 on bowls; 974 ms in PyTorch, 238 ms with vla.cpp on the Orin NX	reproduces
VLA-JEPA [6]	sim	VLA-JEPA trains cheaply on SO-101 data and runs on the robot.	10-trial operator-judged gate (no numeric bar).	9/10 for $9.7; 355 ms per chunk on the Orin NX	reproduces

Table 5: One row per paper with a claim of its own; FAIL-Detect and SAFE share a row. D-JEPA is left out: its only claim was run with proxy labels, not the paper's success labels, so it was never really tested. The bars are ours, written before each test, from the paper's own number where it gives one (that number is kept in the register). Internal names: W1 is our LeWorldModel and W2 our DINO-WM, both trained on our teleoperated data; names such as G1b, H0 or P1 are the ids of our tests in the repository. The rig price of €1,350 counts the parts with a stated price; 2 small parts are not priced yet. Read the verdict against the rig each paper was shown on.

paper	their rig	claim tested	bar	our result	verdict
REKI [21]	sim	Policies without memory oscillate between poses at a measurable rate.	$\geq$ 10 % of failed episodes oscillate (their criterion).	20.3 % of all episodes, 24.0 % of failures; REKI trained on our data: 19/20 + 19/20 on the arm, oscillates in 1/40	reproduces
Real-Time Chunking [4]	unstated	Conditioning a new chunk on the executing one removes the jump at chunk boundaries.	Best gradient-free method cuts the median jump $\geq$ 50 % against naive async, drift $\leq$ +10 %, within 667 ms.	prefix overwrite cuts the jump 71–73 % against naive async at +0.4 ms; tuned LeRobot identity mode is close	not proven
HWM (hierarchical latent world models) [10]	>$20k	A high-level world model over learned macro-actions beats the low-level model beyond about 1 s (their Fig. 6 crossover).	Crossover in [1.0, 2.0] s; at 2 s high-level error $\leq$ 0.95 $\times$ low-level; learned macro-actions $\geq$ net joint displacement on cosine.	no crossover: W1 0.382 $\pm$ 0.005 $\rightarrow$ 0.563 $\pm$ 0.007, high level 0.769 $\pm$ 0.009 $\rightarrow$ 0.759 $\pm$ 0.009 (0.73–2 s); cosine 0.608 $\pm$ 0.007 vs 0.551 $\pm$ 0.006	not proven
LeJEPA identifiability [12]	sim	A SIGReg JEPA trained on goal-directed data recovers the true latent state only partially and unevenly.	W1's mean R^2 from latent to joints $<$ 0.80; anisotropy (max $-$ min per-joint R^2) $\geq$ 0.20; the higher-R^2 model has the higher G1b top-1.	W1 mean R^2 0.565, anisotropy 0.639 (shoulder lift 0.91, wrist flex 0.27); frozen DINOv2-S 0.867 / 0.301	reproduces
SURVEYOR [25]	sim	A threshold on the imagined-vs-reached latent distance accepts good plans and rejects bad ones ($\tau = 0.20$).	τ below the 10th percentile of non-equivalent pairs and $\leq$ 0.40; as a monitor $\geq$ 5/8 failures at $\leq$ 2/42; $\geq$ 0.05 AUROC over a shuffled drafter.	τ = 1.249 on W1 latents of 69 real rollouts; as a monitor 3/8 at 6/42 (S = 30), AUROC 0.756 vs shuffled 0.560	fail (authors' method)
Adaptive Action Chunking (AAC) [11]	>$20k	Chunk length chosen from the entropy of N sampled chunks follows the task's phases (long in transport, short at the grasp), lowers the boundary jump, and costs little with batched sampling.	median h* in transport $\geq$ grasp window + 2; jump with AAC + overwrite $\leq$ fixed + overwrite at d = 8; an N = 20 decision $\leq$ 298 ms on the Orin.	h* 2 in transport and 2 at the grasp (86 % of decisions at the floor); jump 0.973 vs 0.940; N = 20 decision 4,751 ms (sequential) on the Orin	fail (authors' method); the latency claim: our engine (no batched sampling)
PACT-VLA [22]	$5–20k	Picking among N sampled chunks by their flow residual under the pre-commit observation fits what happens next better than a random or stale pick, with no added control-step delay.	PACT beats a random pick and the stale-observation residual at d = 8 (CIs $<$ 0); its delay on the Orin $\leq$ the naive delay; at that delay it still beats a single sample.	offline $-0.27°$ vs random and vs the stale residual (both CIs $<$ 0); on the Orin 1,200 ms per cycle, 37 control ticks vs 8	fail (our engine)
Horizon $\times$ depth (Euler steps vs execution horizon) [23]	>$20k	With long open-loop commitment, cutting a flow-matching VLA's Euler steps from 10 to 2 contracts the action cloud around an unchanged centroid, saves most of the inference time and avoids edge-GPU thermal throttling.	median R(2)/R(10) $\leq$ 0.70 with the centroid within 0.05 relative and 3°; N = 2 latency $\leq$ 0.85 $\times$ N = 10 on the Orin, N = 10 within 5 % of 238 ms.	R(2)/R(10) 0.78 (CI 0.73–0.81), centroid 0.018 and 2.35°; 196.1 vs 237.5 ms per chunk (0.83, vision 151 ms); no throttling at N = 10 on the Orin, at MAXN_SUPER or 25 W	not proven
REMORA [24]	>$20k	Reading a progress distribution out at the supported peak nearest the previous estimate steadies the estimate and lets it fall during real setbacks, with no worse detection of the stages.	Against per-frame argmax: premature not higher and missed $\leq$ +2 points (P1); flips halved, CI upper $\leq$ 0.7 (P2); a decrease inside $\geq$ 50 % of the reversal windows with $\leq$ 20 % false regressions (P3).	premature 0.22 vs 0.25, missed equal (P1 pass); flips 0.70 $\times$ argmax, CI 0.48–0.94 (P2 fail); setback recall 2/7, false 0.52 (P3 fail); MAE 0.184 vs median 0.218	fail (our variant)

Table 5, continued.